

SKILL : WEEK 11

SESSION 21:

Task -1 :

To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components

CODE:

Calculator.h

```
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int subtract(int a, int b);
int multiply(int a, int b);
int divide(int a, int b, int *result);

#endif
```

Calculator.c

```
#include "calculator.h"

int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}

int multiply(int a, int b)
{
    return a * b;
}

int divide(int a, int b, int *result)
{
    if (b == 0)
        return 0;

    *result = a / b;
    return 1;
}
```

Main.c

```
#include <stdio.h>
#include "calculator.h"

int main()
{
    int a, b, choice, result;

    printf("Enter two integers: ");

    if (scanf("%d %d", &a, &b) != 2)
    {
        printf("Error: Invalid input.\n");
        return 1;
    }

    printf("\n1. Add\n");
    printf("2. Subtract\n");
    printf("3. Multiply\n");
    printf("4. Divide\n");
    printf("Enter choice: ");

    if (scanf("%d", &choice) != 1)
    {
        printf("Error: Invalid choice.\n");
        return 1;
    }

    switch (choice)
    {
        case 1:
            result = add(a, b);
            printf("Result = %d\n", result);
            break;

        case 2:
            result = subtract(a, b);
            printf("Result = %d\n", result);
            break;

        case 3:
            result = multiply(a, b);
            printf("Result = %d\n", result);
            break;

        case 4:
            if (divide(a, b, &result))
                printf("Result = %d\n", result);
            else
                printf("Error: Division by zero.\n");
            break;

        default:
            printf("Error: Invalid choice.\n");
    }

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ nano calculator.h
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ nano calculator.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ nano main.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ gcc main.c calculator.c -o calculator
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ ./calculator
Enter two integers: 20 5

1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice: 1
Result = 25
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ ./calculator
Enter two integers: 20 0

1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice: 4
Error: Division by zero.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ ./calculator
Enter two integers: 10 5

1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice: 9
Error: Invalid choice.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ ./calculator
Enter two integers: abd 7
Error: Invalid input.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ |
```

OBSERVATION:

The program successfully demonstrated modular programming and end-to-end execution by connecting main.c, calculator.h, and calculator.c. The functions were declared in the header file and implemented separately in the source file, allowing the modules to communicate correctly. The program successfully performed arithmetic operations and handled invalid input, invalid choices, and division-by-zero conditions without crashing. Thus, module integration, interface validation, error handling, reliability, and component separation were successfully demonstrated.

Task - 2:

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

CODE:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void log_error(const char *message)
{
    FILE *file = fopen("error.log", "a");

    if (file == NULL)
    {
        printf("Error: Could not open log file.\n");
        return;
    }

    fprintf(file, "ERROR: %s\n", message);
    fclose(file);
}

int main()
{
    char command[100];

    printf("Enter a command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    if (strcmp(command, "start") == 0)
    {
        printf("Command executed: START\n");
    }
    else if (strcmp(command, "stop") == 0)
    {
        printf("Command executed: STOP\n");
    }
    else if (strcmp(command, "status") == 0)
    {
        printf("Command executed: STATUS\n");
    }
    else if (strlen(command) == 0)
    {
        printf("Error: Empty command.\n");
        log_error("Empty command entered.");
    }
    else
    {
        printf("Error: Invalid syntax or unknown command.\n");
        log_error("Invalid command entered.");
    }

    printf("Program recovered gracefully.\n");

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ nano error_handler.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ gcc error_handler.c -o error_handler
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ ./error_handler
Enter a command: start
Command executed: START
Program recovered gracefully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ ./error_handler
Enter a command: status
Command executed: STATUS
Program recovered gracefully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ ./error_handler
Enter a command: hello
Error: Invalid syntax or unknown command.
Program recovered gracefully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ cat error.log
ERROR: Invalid command entered.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ ./error_handler
Enter a command:
Error: Empty command.
Program recovered gracefully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ cat error.log
ERROR: Invalid command entered.
ERROR: Empty command entered.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ █
```

OBSERVATION:

The program successfully demonstrated runtime error detection, error message generation, failure logging, and graceful recovery. Valid commands were executed successfully, while invalid commands and empty inputs were detected and reported with appropriate error messages. The detected failures were recorded in the error.log file using append mode, preserving previous error records. The program handled the error conditions without crashing and completed execution normally, demonstrating effective error handling and recovery.

SESSION 22:

Task -1 :

To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

CODE:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *numbers;

    numbers = (int *)malloc(5 * sizeof(int));

    if (numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
    {
        numbers[i] = (i + 1) * 10;
    }

    printf("Allocated memory successfully.\n");

    printf("Values: ");
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", numbers[i]);
    }

    printf("\n");

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ nano memory_leak.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ gcc -g memory_leak.c -o memory_leak
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ ./memory_leak
Allocated memory successfully.
Values: 10 20 30 40 50
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ valgrind --leak-check=full ./memory_leak
==2206== Memcheck, a memory error detector
==2206== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2206== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2206== Command: ./memory_leak
==2206==
Allocated memory successfully.
Values: 10 20 30 40 50
==2206==
==2206== HEAP SUMMARY:
==2206==   in use at exit: 20 bytes in 1 blocks
==2206==   total heap usage: 2 allocs, 1 frees, 1,044 bytes allocated
==2206==
==2206== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==2206==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==2206==    by 0x40011BE: main (memory_leak.c:8)
==2206==
==2206== LEAK SUMMARY:
==2206==   definitely lost: 20 bytes in 1 blocks
==2206==   indirectly lost: 0 bytes in 0 blocks
==2206==   possibly lost: 0 bytes in 0 blocks
==2206==   still reachable: 0 bytes in 0 blocks
==2206==   suppressed: 0 bytes in 0 blocks
==2206==
==2206== For lists of detected and suppressed errors, rerun with: -s
==2206== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$
```

OBSERVATION:

The program was successfully executed and analyzed using Valgrind. The initial Valgrind analysis detected a memory leak of 20 bytes in one block, indicating that dynamically allocated memory had not been released. The leak was traced to the `malloc()` statement in `memory_leak.c`. The issue was identified and corrected by releasing the allocated memory using `free()`. Thus, memory-leak detection, error identification, resource release, and memory cleanup verification were successfully demonstrated.

Task - 2:

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

CODE:

test_script.sh

```
#!/bin/bash

REPORT="test_report.txt"

echo "==== TEST REPORT =====" > $REPORT
echo "Test Program: Addition" >> $REPORT
echo "===== " >> $REPORT

pass=0
fail=0

run_test()
{
    test_name="$1"
    input="$2"
    expected="$3"

    actual=$(echo "$input" | ./test_program)

    echo "Test: $test_name" >> $REPORT
    echo "Expected: $expected" >> $REPORT
    echo "Actual:   $actual" >> $REPORT

    if [ "$actual" = "$expected" ]
    then
        echo "Result: PASS" >> $REPORT
        echo "$test_name: PASS"
        pass=$((pass + 1))
    else
        echo "Result: FAIL" >> $REPORT
        echo "$test_name: FAIL"
        pass=$((pass))
        fail=$((fail + 1))
    fi

    echo "-----" >> $REPORT
}
```

```
run_test "Addition Test 1" "10 20" "Sum = 30"
run_test "Addition Test 2" "5 7" "Sum = 12"
run_test "Addition Test 3" "100 50" "Sum = 150"
run_test "Addition Test 4" "0 10" "Sum = 10"

echo "Total Passed: $pass" >> $REPORT
echo "Total Failed: $fail" >> $REPORT

echo
echo "==== TEST SUMMARY ====="
echo "Passed: $pass"
echo "Failed: $fail"
echo
echo "Report generated: $REPORT"
```

test_program.

```
#include <stdio.h>

int main()
{
    int a, b;

    if (scanf("%d %d", &a, &b) != 2)
    {
        printf("Invalid input\n");
        return 1;
    }

    printf("Sum = %d\n", a + b);

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ nano test_program.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ gcc test_program.c -o test_program
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ echo "10 20" | ./test_program
Sum = 30
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ nano test_script.sh
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ chmod +x test_script.sh
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ ./test_script.sh
Addition Test 1: PASS
Addition Test 2: PASS
Addition Test 3: PASS
Addition Test 4: PASS

==== TEST SUMMARY ====
Passed: 4
Failed: 0

Report generated: test_report.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$ cat test_report.txt
==== TEST REPORT ====
Test Program: Addition
=====
Test: Addition Test 1
Expected: Sum = 30
Actual:   Sum = 30
Result: PASS
-----
Test: Addition Test 2
Expected: Sum = 12
Actual:   Sum = 12
Result: PASS
-----
Test: Addition Test 3
Expected: Sum = 150
Actual:   Sum = 150
Result: PASS
-----
Test: Addition Test 4
Expected: Sum = 10
Actual:   Sum = 10
Result: PASS
-----
Total Passed: 4
Total Failed: 0
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week11$
```

OBSERVATION:

The program successfully demonstrated automated testing and test result verification using a Bash script. Four test cases were designed and executed automatically, and the actual outputs were compared with the expected outputs. All four test cases passed with 0 failures. A detailed test_report.txt file was also generated containing the test cases, expected results, actual results, and final pass/fail count. Thus, test case design, automated execution, output comparison, verification, and report generation were successfully demonstrated.

